

UE : Perl pour l'administration système:

Cours 1 : Les bases de Perl

Dr. ATTA A. Ferdinand

Licence 3 Génie-Informatique/IUA, Année académique 2025-2026

Plan

- 1 Introduction
- 2 Premiers pas en Perl
 - Types de données & Variable
 - Opérateurs
- 3 Structures de contrôle de flux
 - Structures de Décision (SD)
 - Structures Répétitives (SR)
- 4 Conclusion

cours 1 : Objectifs

- Objectifs du cours 1 :
 - Connaître les principes de Perl
 - Savoir écrire des scripts de base en Perl

Qu'est-ce que Perl

- Perl est un langage de programmation **interprété**. (Nous reviendrons bientôt sur ce que cela signifie.)
- Tout langage de programmation est essentiellement un formalisme convivial pour écrire des instructions pour qu'un ordinateur les suive. Ces instructions sont à un moment donné traduites en langage machine, qui est ce que l'ordinateur "comprend" réellement.

Langages de programmation vs. langages humains (1/3)

- Puisque nous sommes tous des linguistes, il est intéressant de réfléchir un instant aux différences entre les langages informatiques et les langues "naturelles".
 - La plupart des langages de programmation possèdent une syntaxe récursive, et peuvent donc théoriquement générer des structures infiniment complexes infiniment complexes, comme le langage humain.
 - Les langages de programmation ont une "syntaxe" et une "sémantique", et quelque chose qui s'apparente à des classes de mots.
 - Cependant, les langages de programmation ont des lexiques extrêmement réduits, et des règles syntaxiques très restreintes.
 - Une autre grande différence est que les langages informatiques traitent des domaines très restreints par rapport aux langues humains, qui sont beaucoup plus généraux et flexibles.

Langages de programmation vs. langages humains (2/3)

- Il est important de noter que les programmes informatiques ne peuvent pas tolérer l'ambiguïté - les programmes doivent être absolument précis.
- La pragmatique n'entre pas vraiment en ligne de compte ; un ordinateur ne peut pas "comprendre ce que vous voulez dire" en dépit de ce que vous dites (c'est-à-dire sans implicature).
- S'il existe un contre-exemple à cela, c'est probablement Perl, qui est extrêmement habile à "faire ce qu'il faut", avec une quantité limitée d'explicitation dans les instructions.
- De plus, Perl (comme Python et quelques autres langages) a été conçu pour ressembler le plus possible à l'anglais, il est donc relativement convivial.

Langages de programmation vs. langages humains (3/3)

- Une grande différence entre l'écriture de programmes et l'écriture d'autres types de documents est que vos programmes doivent avoir une syntaxe parfaite, sinon ils ne fonctionneront pas. Chaque petite virgule et chaque point-virgule sont essentiels.
- Par conséquent, en écrivant du code, il est vital d'être absolument méticuleux. Sinon, vous allez pleurer, grincer des dents et, de manière générale, ne pas vous amuser beaucoup.
- Le bon côté des choses, c'est que la syntaxe est en fait très facile. Beaucoup, beaucoup plus facile que dans n'importe quelle langue humaine.
- Encore mieux, Perl est un langage très indulgent qui permet de multiples formes d'expression. Cela signifie que vous pouvez dire quelque chose de différentes manières, tant que votre syntaxe est correcte.

Perl vs. autres langages de programmation

- La plupart des langages de programmation tels que C, C++ sont **des langages compilés**.
 - Pour exécuter un programme, vous créez un document texte avec le code, vous exécutez un compilateur dessus pour le convertir en code machine pour votre système d'exploitation, puis vous l'exécutez.
- Perl est un **langage interprété**, comme Java, Python.
 - Pour exécuter un programme dans un tel langage, vous créez un document texte et indiquez à l'interpréteur (ou à la machine virtuelle) de l'exécuter en tant que programme. L'interprète le vérifie, le compile en code machine et l'exécute.
 - Le processus en deux étapes des langages interprétés les rend légèrement plus facile à utiliser. Vous pouvez constamment vérifier votre code en l'exécutant après chaque modification.

Historique de Perl

- Perl a été conçu au milieu des années 1987 par Larry Wall, alors programmeur chez Unisys : Perl v1.0 sort le 18 Décembre 1987.
- Il a combiné des caractéristiques utiles de plusieurs langages existants avec une syntaxe conçue pour ressembler le plus possible à l'anglais.
- Depuis lors, Perl s'est transformé en un langage puissant et populaire, avec de nombreux modules fournis par la communauté open-source.
- Perl est conçu pour être flexible, intuitif, facile et rapide ; ce qui le rend quelque peu "désordonné".
- Perl est en fait un ensemble de deux langages de programmation - **Perl 5** et **Perl 6**. Lorsque nous disons simplement "Perl", nous nous référons à Perl 5. En octobre 2019, Perl 6 a été officiellement renommé **Raku**.
- Dans ce cours, nous traiterons de Perl 5 appelé Perl.

Culture de Perl

- À bien des égards, Perl est un langage pour les "hackers", et non pour les informaticiens. Perl est dédaigné par les personnes qui préfèrent des langages comme le C, plus rigides et plus proches du code machine.
- Le point de vue de Larry Wall est le suivant :
 - "Perl est un langage qui permet de faire son travail".
- Deux autres slogans :
 - "Les choses faciles devraient être faciles, et les choses difficiles [...] devraient être possibles."
 - "Il y a plus d'une façon de faire".

Champ d'application perl

- Perl est un langage de programmation multiparadigme de haut niveau à usage général et multiplateforme :
 - Application de Bureau (PerlQt, PerlTk, etc.)
 - Application web (catalyst, perl Dancer, etc.)
 - Accès et gestion de BD (MySQL, MongoDB, Oracle, etc.)
 - Applications réseau
 - Application scientifique
 - Administration système(automatisation de tâches, etc.)
 - Etc.

Installation

■ Interpréteur Perl

- <https://www.perl.org/get.html>
- Pour vérifier ... : ouvrir CMD puis saisir : `perl -v`

■ Editeur de code

- Notepad++, Komodo IDE, Padre

Exécution de script

- **Depuis la ligne de commande :** `perl -e "mon_code"`

```
C:\>perl -e "print 'Hello, World!'"
Hello, World!
C:\>
```

- **Depuis un fichier .pl :**

- créer et enregistrer un fichier .pl : `monscript.pl`

```
1  #!/usr/bin/perl
2  print 'Hello, World!';
```

- `C:\> perl monscript.pl`

- **Depuis un IDE (ex : Komodo, padre)**

Types de données

- **Un type de données** est utilisé pour comprendre la catégorie de données avec laquelle nous travaillons.
- **Une variable** est un nom donné à un emplacement mémoire.
- Perl propose trois catégories types de données **scalaires, les tableaux et les tables de hachage.**

Types de données

- **Scalaire** : Les variables scalaires sont utilisées pour stocker des nombres et des chaînes de caractères. Les nombres peuvent être des entiers, des valeurs à virgule flottante, des valeurs hexadécimales et octales. Les noms des variables scalaires commencent par **un signe de dollar (\$)**.
- **Tableau** : Un tableau est une collection d'éléments. Les variables d'un tableau commencent par **le signe arobase (@)**.
- **Table de hachage** : Les tables de hachage sont utilisées pour stocker des données à l'aide de paires clé-valeur. Les variables de hachage commencent par **le signe de pourcentage (%)**.

Notion de variable

Définition

Une variable est un identifiant utilisé pour adresser un emplacement de mémoire. Lorsqu'une variable est déclarée, une certaine quantité de mémoire lui est réservée pour stocker son contenu. Cet emplacement mémoire peut être adressé de manière unique à l'aide de son adresse mémoire. Comme il serait difficile pour les programmeurs de se souvenir de l'emplacement mémoire chaque fois qu'ils accèdent à des données à partir d'un emplacement mémoire, il existe un concept de variables. Dans cette section, nous ne parlerons que des variables scalaires. Un cours dédié aux tableaux et aux hachages est prévu.

Déclaration de variable

- *\$nom_variable = valeur_initiale ;*
- Suites de caractères alphanumériques et quelques caractères spéciaux
- Mots réservés du langage interdit comme : **for, print, while, if**
- Un nom ne doit pas commencer par un chiffre
- **Affectation** ou **assignation** ("=") $\Rightarrow$ établissement de lien entre le nom de la variable et sa valeur (son contenu)

```
1 $name = "Rose"; #String
2 $age = 29;      #Integer
3 $weight = 45.74; #Float
```

Affichage de variable

```
1 $name = "Jody";
2 $company = "Microsoft";
3 $package = 120;
4 print ($name); #Afficher une seule variable
5 #Afficher plusieurs variables séparées par virgule.
6 #Aucun espace ne sera laissé lors de l'affichage de ces variables.
7 print ($name, $company, $package);
8 #Afficher plusieurs variables séparées par virgule.
9 #Insertion d'espaces entre les variables.
10 print ($name, " ", $company, " ", $package);
11 #Faire des variables un élément d'une chaîne plus large
12 print ("\nName: $name \nCompany = $company \nPackage =
13 $package \n");
```

Interaction avec les utilisateurs

- Afin d'accepter les entrées d'un utilisateur, nous utilisons l'invite ; donnée par <STDIN> ou simplement <>. Une variable scalaire est nécessaire pour recevoir l'entrée provenant de l'utilisateur. Syntaxe générale :
`$nom_variable = <STDIN>; #nom_variable = <>;`

```
1 use strict;
2 use warnings;
3 print ("\nEntrez un nom: ");
4 #Utiliser l'invite pour recevoir les données dans
5 #la variable $nom
6 my $nom = <STDIN>;
7 print ("\nEntrez de l'age ");
8 #Utiliser l'invite pour recevoir les données dans $age
9 my $age = <STDIN>;
10 #Afficher le contenu de $nom et $age
11 print ("\n Vous avez saisi:\n");
12 print ("Nom", " ", $nom, " ", "Age", " ", $age);
```

Interaction avec les utilisateurs

- `"\n"` est ajouté à notre entrée chaque fois que nous avons saisi quelque chose et appuyé sur Entrée. Laisse une ligne supplémentaire. Pour contourner ce problème, nous pouvons utiliser la fonction **chomp** qui supprimera le caractère de retour à la ligne. Elle peut être utilisée soit lors de la saisie avec `<>` / `<STDIN>`, soit une fois que la saisie est reçue dans une variable.

```
1 use strict;
2 use warnings;
3 print ("\nEntrez un nom: ");
4 #Utiliser l'invite pour recevoir les données dans $nom
5 chomp(my $nom = <STDIN>);
6 print ("\nEntrez de l'age ");
7 #Utiliser l'invite pour recevoir les données dans $age
8 chomp(my $age = <STDIN>);
9 #Afficher le contenu de $nom & $age
10 print ("\n Vous avez saisi:\n");
11 print ("Nom", " ", $nom, " ", "Age", " ", $age);
```

Arithmétiques

Operator	Description	Sample Usage	Explanation
+	Addition	\$a + \$b	Performs addition and returns the sum of all the operands.
-	Subtraction	\$a - \$b	Subtracts the operand on the right from the operand on the left.
*	Multiplication	\$a * \$b	Multiplies operands and returns the product.
/	Division	\$a / \$b	Divides the operand on the left by the operand on the right and returns the quotient.
%	Modulus	\$a % \$b	Divides the operand on the left by the operand on the right and returns the remainder. This operator will work correctly only with integers.
**	Exponent	\$a ** \$b	Raises the power of the operand on the left by a value specified by the operand on the right. Eg. $5^{**}2$ is equivalent to 5^2 in mathematical form which is equal to 25.
++	Increment	\$a ++	Increments the value of the given variable by one. \$a++ is equivalent to \$a = \$a + 1.
--	Decrement	\$b --	Decrements the value of the given variable by one. \$b-- is equivalent to \$b = \$b - 1.

Assignations composées

Operator	Description	Sample Usage	Equivalent Expression
<code>+=</code>	Add operands, assign sum to the operand on the left.	<code>\$a += \$b</code>	<code>\$a = \$a + \$b</code>
<code>-=</code>	Subtract the operand on the right from the one on the left and assign the difference to the left operand.	<code>\$a -= \$b</code>	<code>\$a = \$a - \$b</code>
<code>*=</code>	Perform multiplication and assign the product to the operand on the left.	<code>\$a *= \$b</code>	<code>\$a = \$a * \$b</code>
<code>/=</code>	Divide the left operand by the right one and assign the quotient to the left operand.	<code>\$a /= \$b</code>	<code>\$a = \$a / \$b</code>
<code>%=</code>	Divide the left operand by the right one and assign the remainder to the left operand.	<code>\$a %= \$b</code>	<code>\$a = \$a % \$b</code>
<code>**=</code>	Calculate exponent and assign the value to the left operand.	<code>\$a **= \$b</code>	<code>\$a = \$a ** \$b</code>

Logiques

Operator	Description	Sample Usage	Explanation
and #Alternative# &&	Logical AND	(Expr1) and (Expr2) #Alternative# (Expr1) && (Expr2)	Returns <i>True</i> if all the expressions evaluate to <i>True</i> , <i>False</i> otherwise.
or #Alternative# 	Logical OR	(Expr1) or (Expr2) #Alternative# (Expr1) (Expr2)	Returns <i>True</i> if any one of the expressions evaluates to <i>True</i> . Returns <i>False</i> if all the expressions evaluate to <i>False</i> .
!	Logical NOT	! (Expr)	Inverts the result. If the expression evaluates to <i>False</i> , <i>True</i> will be returned and if it evaluates to <i>True</i> , <i>False</i> will be returned.

Comparaisons

Operator	Description	Sample Usage	Explanation
==	Equal To	\$a == \$b	Returns True if both the operands are <i>equal</i> , False otherwise.
!=	Not Equal To	\$a != \$b	Returns True if both the operands are <i>NOT equal</i> , False otherwise.
<	Less Than	\$a < \$b	Returns True if the value of the operand on the left <i>is less than</i> the value of the operand on the right, False otherwise.
>	Greater Than	\$a > \$b	Returns True if the value of the operand on the left <i>is greater than</i> the value of the operand on the right, False otherwise.
<=	Less Than OR Equal To	\$a <= \$b	Returns True if the value of the operand on the left <i>is less than OR equal to</i> the value of the operand on the right, False otherwise.
>=	Greater Than OR Equal To	\$a >= \$b	Returns True if the value of the operand on the left <i>is greater than OR equal to</i> the value of the operand on the right, False otherwise.

Binaire

Operator	Description	Sample Usage	Explanation
&	Bitwise Logical AND	\$a & \$b	Performs logical AND on each corresponding bit of the operands.
	Bitwise Logical OR	\$a \$b	Performs logical OR on each corresponding bit of the operands.
~	Bitwise Logical Inverter	~\$a	Computes binary one's compliment.
^	Bitwise Logical XOR	\$a ^ \$b	Performs logical XOR on each corresponding bit of the operands.
<<	Left Shift	\$a << \$b	Left shifts bits of the operand on the left as many times as specified by the operand on the right. Eg. $2 \ll 3$ will left shift 2's bits (10 in binary) 3 times. So, $2 \ll 3$ will be equal to 16 (10000 in binary)
>>	Right Shift	\$a >> \$b	Right shifts bits of the operand on the left as many times as specified by the operand on the right. Eg. $7 \gg 1$ will right shift 7's bits (111 in binary) 1 time. So, $7 \gg 1$ will be equal to 3 (11 in binary).

Types de structures

Structures de contrôle

Les structures de contrôle permettent de contrôler l'exécution des instructions dans le code. Deux types de structures sont disponibles :

- **Les Structures de Décision (SD) ou conditionnelles :**
Elles aiguillent l'exécution du code en fonction des valeurs que pourra prendre un test.
- **Les Structures de Répétition (SR) :** Elles font exécuter une portion de code un certain nombre de fois, jusqu'à ce que une condition soit remplie ou tant qu'une condition est remplie

Typologie de SD : conditionnelle simple (1/2)

Conditionnelle simple

Si <condition> alors <séquence>

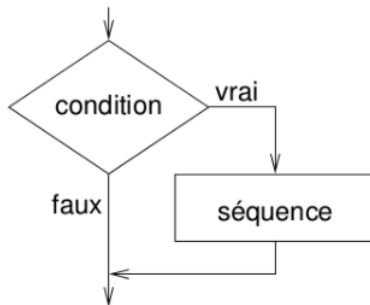


Figure 1 – Structure conditionnelle simple

Typologie de SD : conditionnelle simple (2/2)

Si .. alors : if ... :

Syntaxe

```
if (<condition>){  
    #Code à exécuter si <condition> est vraie  
}
```

```
1 use strict;  
2 use warnings;  
3 print ("Entrez votre age: ");  
4 chomp(my $age = <>);  
5 if ($age <= 7){  
6     print("Vous êtes un enfant!!! ");  
7 }
```

Typologie de SD : conditionnelle composée (1/3)

Structure conditionnelle composée

Si <condition> alors <séquence1> Sinon <séquence2>.

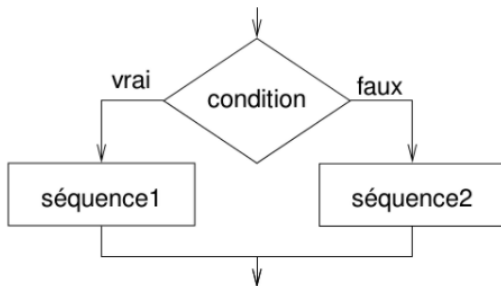


Figure 2 – Structure conditionnelle Composée

Typologie de SD : conditionnelle composée (2/3)

Si ... sinon : if ... else

Syntaxe

```
if (<condition>){  
    #Code à exécuter si <condition> est vraie  
}else{  
    #Code à exécuter si <condition> est faux  
}
```

Typologie de SD : conditionnelle composée (3/3)

Si ... sinon : if ... else

```
1 use strict;
2 use warnings;
3 print ("Entrez votre age: ");
4 chomp(my $age = <>);
5 if ($age <= 7){
6     print("Vous êtes un enfant!!! ");
7 }else{
8     print("Vous n'êtes pas un enfant!!! ");
9 }
```

Typologie de SD : conditionnelle multiple (1/3)

Structure conditionnelle multiple

Si <condition1> alors <séquence1> Sinon Si <condition2> alors <séquence2> ... Sinon <séquenceN>.

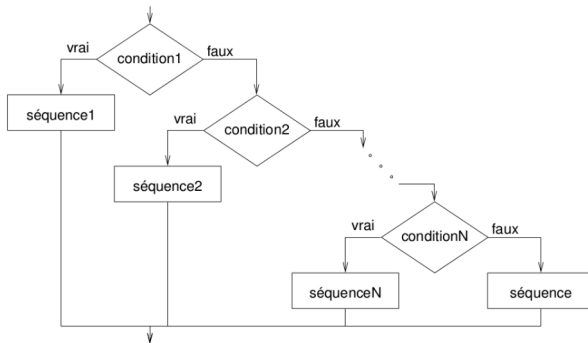


Figure 3 – Structure conditionnelle Multiple

Typologie de SD : conditionnelle multiple (2/3)

Si .. sinonsi ... else... : if ... elif ... else

Syntaxe

```
if (<condition1>){
    #Code à exécuter si <condition1> est vraie
}elseif(<condition2>){
    #Code à exécuter si <condition1> est faux et <condition2>
    #est vraie
}
...
else{
    #Code à exécuter si <condition1>,..., <conditionN>
    #sont fausses
}
```

Typologie de SD : conditionnelle multiple (3/3)

Si .. sinon si ... else... : if ... elsif ... else

```
1 use strict;
2 use warnings;
3 print ("Entrez votre age: ");
4 chomp(my $age = <>);
5 if ($age >= 45){
6     print("Vous êtes un adulte");
7 }elsif ($age >= 18){
8     print("Vous êtes un jeune!!! ");
9 }elsif ($age >= 8){
10    print("Vous êtes un adolescent!!! ");
11 }else{
12    print("Vous êtes un gamin");
13 }
```

N.B : Il existe également *unless* (<condition>) qui s'exécute si la condition est fausse

UE : Perl pour l'administration système:, Cours 1 :

given-when (1/3)

- En Perl, l'instruction **given-when** remplace les longues instructions if qui comparent une variable à de nombreuses valeurs intégrales.
 - L'instruction **given-when** est une instruction de branchement qui offre de nombreuses options. Elle nous permet d'acheminer facilement l'exécution vers différentes zones du code en fonction de la valeur de l'expression.
 - Une instruction de contrôle est fournie pour permettre à une valeur de modifier le contrôle de l'exécution.
- L'instruction switch-case en Perl est analogue à l'instruction switch-case en C/C++, Python ou PHP. Elle aussi remplace de nombreuses instructions if par des cas distincts, tout comme l'instruction switch.

given-when (2/3)

Syntaxe

```
given(expression)
{
    when(value1) {Statement;}
    when(value2) {Statement;}
    default {# Code if no other case matchs}
}
```

given-when (3/3)

```
1 use strict;
2 use warnings;
3 print "Entrez un code de jour entre 0-6: ";
4 chomp(my $daycode = <>);
5 given ($daycode){
6   when ('0') { print 'Sunday' ;}
7   when ('1') { print 'Monday' ;}
8   when ('2') { print 'Tuesday' ;}
9   when ('3') { print 'Wednesday' ;}
10  when ('4') { print 'Thursday' ;}
11  when ('5') { print 'Friday' ;}
12  when ('6') { print 'Saturday' ;}
13  default { print 'code Invalide';}}
```

Structures répétitives : boucle "POUR" (1/2)

Caractéristiques

La structure répétitive "POUR" est utilisée dans le cas où on a un bloc d'instructions à répéter n fois, avec n connu à l'avance, c-à-dire avant l'exécution de la boucle "POUR"

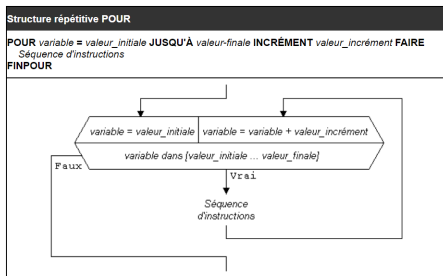


Figure 4 – Boucle "POUR"

UE : Perl pour l'administration système:, Cours 1 :

Les bases de Perl

Structures répétitives : boucle "POUR" (2/2)

For ...

Syntaxe classique

```
for (<variable initialization>;<condition>;<increment/decrement>)  
{  
#Code à exécuter tant que la <condition> est vraie.  
}
```

```
1 # Exemple: Affiche les températures 0C, ..., 10C,  
2 #en Fahrenheit  
3 for($celsuis=0; $celsuis <=10;$celsuis++){  
4     $fahrenheit = ($celsuis * 9)/5 + 32;  
5     print("\n$celsuis C =  $fahrenheit F");  
6 }
```

Structures répétitives : boucle "Tant Que" (1/3)

Caractéristiques

La structure répétitive "Tant Que" est utilisée dans le cas où on a un bloc d'instructions à répéter et le nombre de répétitions n'est pas connu à l'avance, c-à-dire avant l'exécution de la boucle "Tant Que".

Cependant, on doit pouvoir dans ce cas, déterminer quelle ou quelles sont les conditions de répétitions

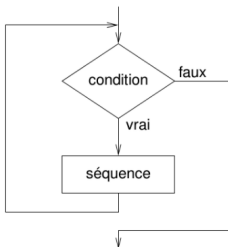


Figure 5 – Boucle "Tant Que"

UE : Perl pour l'administration système:, Cours 1 :

Les bases de Perl

Structures répétitives : boucle "Tant Que" (2/3)

Boucle Tant que : while

Syntaxe classique

```
while (<condition>){  
    #Code à exécuter si <condition> est vraie  
}
```

Structures répétitives : boucle "Tant Que" (3/3)

Boucle Tant que : while

```
1  use strict;
2  use warnings;
3  $number = 1;
4  #Itérer de 1 à 10
5  while ($number <= 10)
6  {
7      #Afficher number
8      print ("\n$number");
9      #Incrémenter number
10     $number++;
11 }
12 print ("\n");.
```

Structures répétitives : boucle "Faire ... Tant Que" (1/2)

Boucle Tant que :do ... while

La boucle "do-while" est similaire à la boucle "while" et continue d'exécuter le bloc de la boucle tant que la condition donnée est vraie.

Syntaxe classique

```
do{  
    #Code à exécuter tant que <condition> est vraie.  
}while (<condition>);
```

Structures répétitives : boucle "Faire ... Tant Que" (2/2)

Boucle Tant que : while

```
1 use strict;
2 use warnings;
3 $number = 10;
4 #Itérer de 10 à 1, vérifier la condition à la fin.
5 do
6 {
7 #Afficher number
8 print ("\n$number");
9 #Décrémente number
10 $number--;
11 } while($number >= 1);
12 print ("\n");
```

Structures répétitives : Until (1/2)

Contrairement aux boucles while et do-while, la boucle until poursuit l'exécution du bloc de boucle tant que la condition donnée n'est pas vraie.

Syntaxe classique

```
until (<condition>){  
    #Code à exécuter tant que <condition> est fausse.  
}
```

Structures répétitives : Until(2/2)

```
1 use strict;
2 use warnings;
3 $number = 1;
4 #Itérer de 10 à 1, vérifier la condition à la fin.
5 until ($number == 11)
6 {
7   #Afficher number
8   print ($number * 5);
9   print ("\n");
10  #Incrémenter number
11  $number++;
12 }
```

Resumons

- Perl est un puissant et flexible langage
- La suite :
 - Structures particulières en Perl
 - TP1
 - Programmation Modulaire en Perl
 - TP2
 - Applications
 - Projet Etudiant

References

- [1] Nathan Metzler, *Perl Programming for Beginners*, 2020.
- [2] Tom Christiansen, brian d foy, Larry Wall, Jon Orwant, *Programming Perl : Unmatched Power for Text Processing and Scripting*, O'Reilly Media, 2012.
- [3] Sufyan bin Uzayr, *Mastering Perl*, CRC Press, 2023